

Vikram Thirumaran

October 10th, 2025

CS 471/ Thanh Nguyen

Fall 2025

Written Assignment 1

Deadline: Oct 15th, 2025

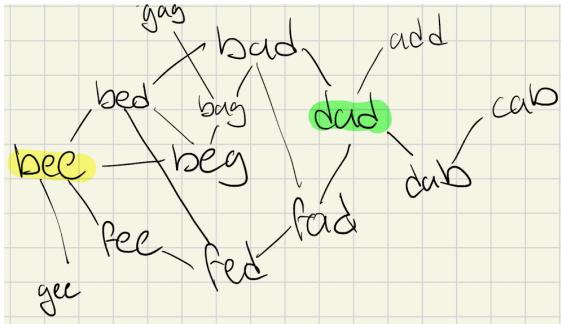
Instruction: You may discuss these problems with classmates, but please complete the write ups individually. (This applies to BOTH undergraduates and graduate students.) Remember the collaboration guidelines set forth in class: you may meet to discuss problems with classmates, but you may not take any written notes (or electronic notes, or photos, etc.) away from the meeting. Your answers must be typewritten, except for figures or diagrams, which may be hand-drawn.

Q1. Word Ladder (30 points)

Wikipedia describes the rules for the word game Word Ladder as: The player is given a start word and an end word (of equal length). In order to win the game, the player must change the start word into the end word progressively, creating an existing word at each step. Each step consists of replacing a single letter. For example, the following is a solution to the Word Ladder puzzle between words “Cold” and “Warm”: COLD → CORD → CARD → WARD → WARM

1. (10 pts) Formulate the problem of solving word ladders as a search problem.
 - **State space:** All valid words in the dictionary
 - **Start State:** The initial given word from the puzzle.
 - **Successor Function:** This is the most critical part. For any given state (word w), the successor function generates a set of new states by applying valid actions.
 - **Actions:** Change one letter at a specific position in the word. For a four-letter word, there are $4 * 25$ possible single-letter changes.
 - **Costs:** Limited to one word change, each action has an equal cost of “1”
 - **Successors:** The resulting words from an action **if and only if** they are valid English words found in our dictionary.
 - **Goal test:** Final word == desired word

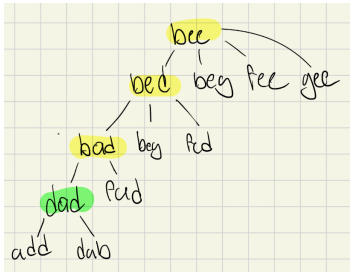
2. (10 pts) Consider a restriction where the words can only use the first seven letters (a-g). Draw the state space graph for converting “bee” to “dad”.



3. (10 pts) (Tree Search) Which part of the search tree is expanded by DFS?

Depth-First Search (DFS) expands what is known as a "left prefix" of the search tree. The algorithm's core strategy is to always explore the deepest unexpanded node first, which it accomplishes by using a LIFO (Last-In, First-Out) stack for its fringe. A tie-breaking rule, such as choosing successors in alphabetical order, makes the search deterministic and creates a "leftmost" path search.

The most important feature of DFS is that when it reaches a dead end (a node that isn't the goal and has no more children to explore), it backtracks to a previous node to explore an alternate branch. This process of diving deep, backtracking, and then diving deep down a new branch is what reveals the tree's structure and allows the algorithm to be complete. Therefore, even though a simple search might find a solution on its first try, this backtracking mechanism is fundamental to the algorithm's general behavior and defines its exploration pattern. It is important to consider however, that the search tree may look different based on the bias of the DFS however. In this case, my layout makes an alphabetical & a left > right DFS search look the same but for a different word or state space layout may change this!. The search tree for the above problem + state space graph would look like this if it was in a “left biased” DFS:



Q2. Graph Search (50 points)

Q2.1. Search Algorithms. (30 points) Consider the following graph. The start node is S and the goal node is G . For the following sub-questions, ties are broken in alphabetical order.

1. (10 pts) In what order are states expanded by UCS? What path would UCS return?

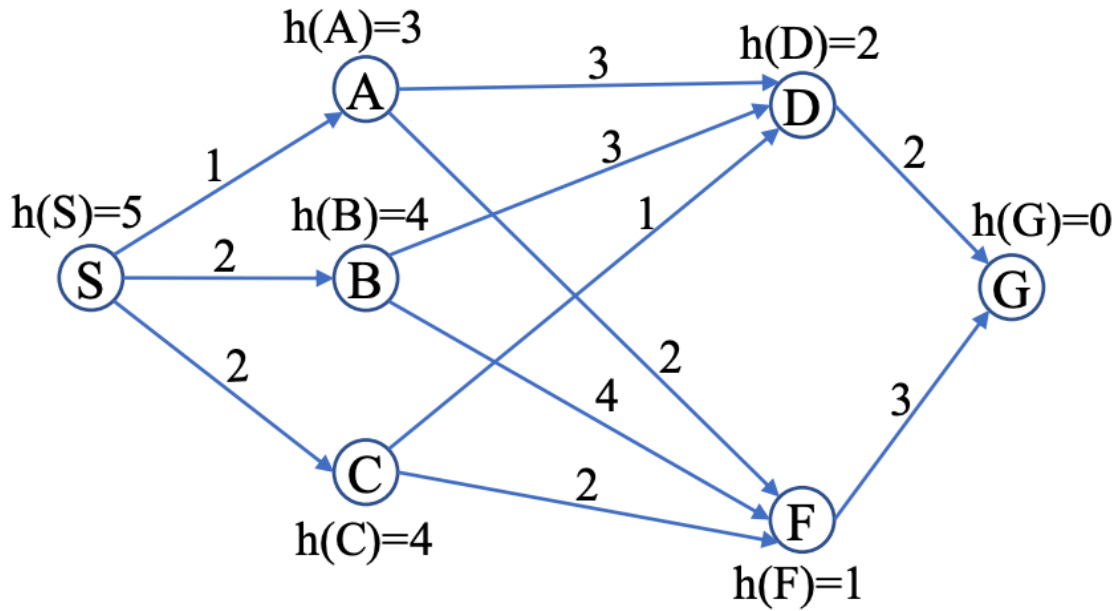
- Order of states expanded: $S > A > B > C > D > F > G$
- Path returned: $S > C > D > G$
- Total Cost: 5

2. (10 pts) In what order are states expanded by Greedy? What path would Greedy return?

- Order of states expanded: $S > A > F > G$
- Path returned: $S > A > F > G$
- Total Cost: 6

3. (10 pts) In what order are states expanded by A^* ? What path would A^* return.

- Order of states expanded: $S > A > F > B > C > D > G$
- Path returned: $S > C > D > G$
- Total Cost: 5



Q2.2 Admissibility/Consistency. (20 point).

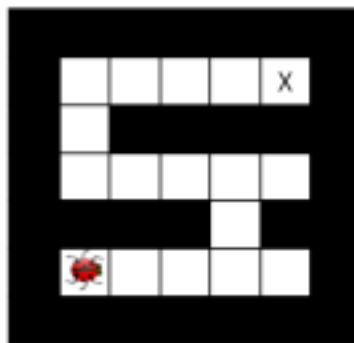
- (10 pts) Is the above heuristic function h admissible? Explain your answer.
- (10 pts) For which TWO states (S, A, B, C, D, F, or G) could you change the heuristic values to make everything consistent? What range of values are possible to make this correction?

Important note. For graph search, we never expand a state type twice.

The heuristic is **not admissible** because it overestimates the cost for node C, where $h(C) = 4$ but the true optimal cost to the goal, $h^*(C)$, is 3 via path $C > D > G$. It is also not consistent, with violations occurring at states S and C. For instance, the path $S > A$ is inconsistent because $h(S) = 5$ is greater than $\text{cost}(S,A) + h(A) = 4$. To make the heuristic consistent, the value for $h(S)$ would need to be in the range $[0, 4]$, and the value for $h(C)$ would need to be in the range $[0, 3]$. In other words, the difference between the heuristic cost and the true costs cannot be greater than the true cost.

Q3. Hive Minds: Lonely Bug (20 points)

You control a single insect in a rectangular maze-like environment with dimensions $M \times N$. Figure below is an example of such an environment. The insect must reach a designated target location X , also known as the hive. There are no other insects moving around. At each time step, an insect can either (a) move into an adjacent square if that square is currently free, or (b) stay in its current location. Squares may be blocked by walls, but the map is known. Optimality is always in terms of time steps; all actions have cost 1 regardless of the number of insects moving or where they move.



1. (10 pts) Provide a minimal correct state space representation? What is the state-space size?

2

Important note. Your computation should be for the general rectangular maze-like environment (not for the example provided).

- **Minimal correct state space** representation for the insect is its (x, y) coordinate on the $m * n$ grid. This covers all cases because the map, with the walls, goal location & start location is static, and only has the single moving bug entity to account for, meaning position is the only variable.
- **State-Space Size:** The size of the state space is the total number of squares the insect can occupy. For a general $m * n$ environment with a number of blocked (wall) squares W , the size is $m * n - W$

2. (10 pts) Provide two heuristics which are admissible.

- a. **Manhattan Distance:** This is the most common and effective heuristic for grid-based movement. If the insect is at (x_1, y_1) and the hive is at (x_2, y_2) , the heuristic is calculated as: $h(n) = |x_1 - x_2| + |y_1 - y_2|$. This is admissible because it represents the absolute minimum number of moves required to reach the goal if there were no walls. Since walls can only make the actual path longer, the Manhattan distance will never be greater than the true shortest path cost.
- b. **The "Zero" Heuristic:** A trivial but always admissible heuristic is: $h(n) = 0$. This is admissible because the true cost to reach the goal can never be negative, so this heuristic never overestimates the cost. Using this heuristic with A* search effectively turns the algorithm into Uniform-Cost Search (UCS).